

# MATLAB Simulation Exercise

## Statistical Signal Processing II

### **RLS ALGORITHM**

Group 09

Student 1: BRIAN KIPROP KIBOR 2508789

Student 2: SUDIP SAHA 2508788

Student 3: SHAMIM AHMED 2509740

University of Oulu

Finland

February 28, 2026

## Theoretical Background

In a wireless/digital communication link, the transmitted symbols pass through a multipath channel. A multipath channel behaves like a short FIR filter, so each received sample contains contributions from several neighboring transmitted symbols. This creates inter-symbol-interference (ISI), which makes symbol detection harder and increases the symbol error rate (SER). [1]

Let the received signal be  $u[n]$ . Because the channel has memory,  $u[n]$  is a mixture of the current symbol and previous symbols (ISI), plus noise.

A **linear FIR equalizer** tries to “undo” the channel by filtering the received samples with a set of adjustable coefficients [1]:

- The equalizer takes a vector of recent samples  $[u[n], u[n - 1], \dots, u[n - L + 1]]$
- It produces an output  $y[n]$ , which should be close to the original transmitted symbol.

During training, we know the correct transmitted symbol  $d[n]$ . The equalizer output error is:

$$e[n] = d[n] - y[n]$$

The goal here is to adjust the equalizer coefficients so that this error becomes small. It can be achieved with LMS adaptive equalizer or RLS adaptive equalizer.

### **LMS (Least Mean Squares) equalization:**

The LMS algorithm is a low-complexity gradient-based method that approximates steepest descent on the mean-square error. With step size  $\mu$ , the LMS weight update is

$$w[n + 1] = w[n] + \mu u[n] e^*[n]$$

where  $(.)^*$  denotes complex conjugate. LMS is simple to implement but its convergence speed depends strongly on  $\mu$  and on the eigenvalue spread of the input correlation matrix. Typically, LMS converge more slowly and may show steady-state fluctuation due to the constant step size [2].

### **RLS (Recursive Least Squares) equalization:**

RLS is an alternative adaptive method that minimizes an exponentially weighted least-squares cost:

$$J[n] = \sum_{k=0}^n \lambda^{n-k} |e[k]|^2$$

where  $0 < \lambda \leq 1$  is the forgetting factor. When  $\lambda = 1$ , all past errors are weighted equally; when  $\lambda < 1$ , older samples are gradually forgotten, which improves tracking in time-varying conditions but may increase sensitivity to noise in a static channel. [1]

RLS uses the inverse correlation matrix  $\mathbf{P}[n]$  to compute a gain vector and update the weights efficiently [1]:

$$k[n] = \frac{P[n-1]u[n]}{\lambda + u^H[n]P[n-1]u[n]}$$

$$w[n] = w[n-1] + k[n]e^*[n]$$

$$P[n] = \frac{1}{\lambda} (P[n-1] - k[n]u^H[n]P[n-1])$$

Compared to LMS, RLS typically achieves much faster convergence because it uses second order correlation information through  $\mathbf{P}[n]$ , at the cost of higher computational complexity.

### Problem Setup

In this experiment, adaptive filtering is used for channel equalization in a digital communication link. A QPSK symbol sequence is transmitted through a 5-tap FIR delay-line channel, which introduces inter-symbol-interference (ISI) because each received sample becomes a weighted sum of multiple transmitted symbols. Additive noise is also present, and the signal-to-noise ratio is set to 25dB. To reduce ISI before demodulation, the receiver employs an L-tap transversal (smoothing-type) adaptive equalizer with length  $L = 20$ , as illustrated in the below block diagram.

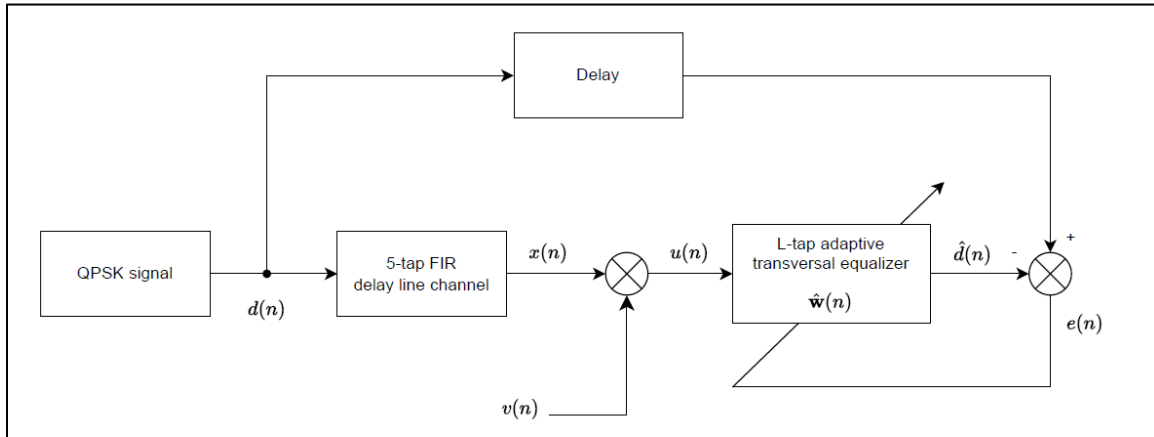


Figure 1. Block diagram of an adaptive channel equalization

## Results and Discussion

**Task 1:** Write the Matlab code for an RLS adaptive filter to equalize the received signals. The forgetting parameter is set to  $\lambda = 1$ . Compare the performance, in terms of convergence and symbol error rate (SER), between the RLS and LMS algorithms.

We have completed the code given and will be returning with the report. Below are the plots, brief explanation of the plots and a discussion for comparison in terms of performance, convergence and symbol error rate (SER).

LMS SER: 0.0077

RLS SER: 0.0012

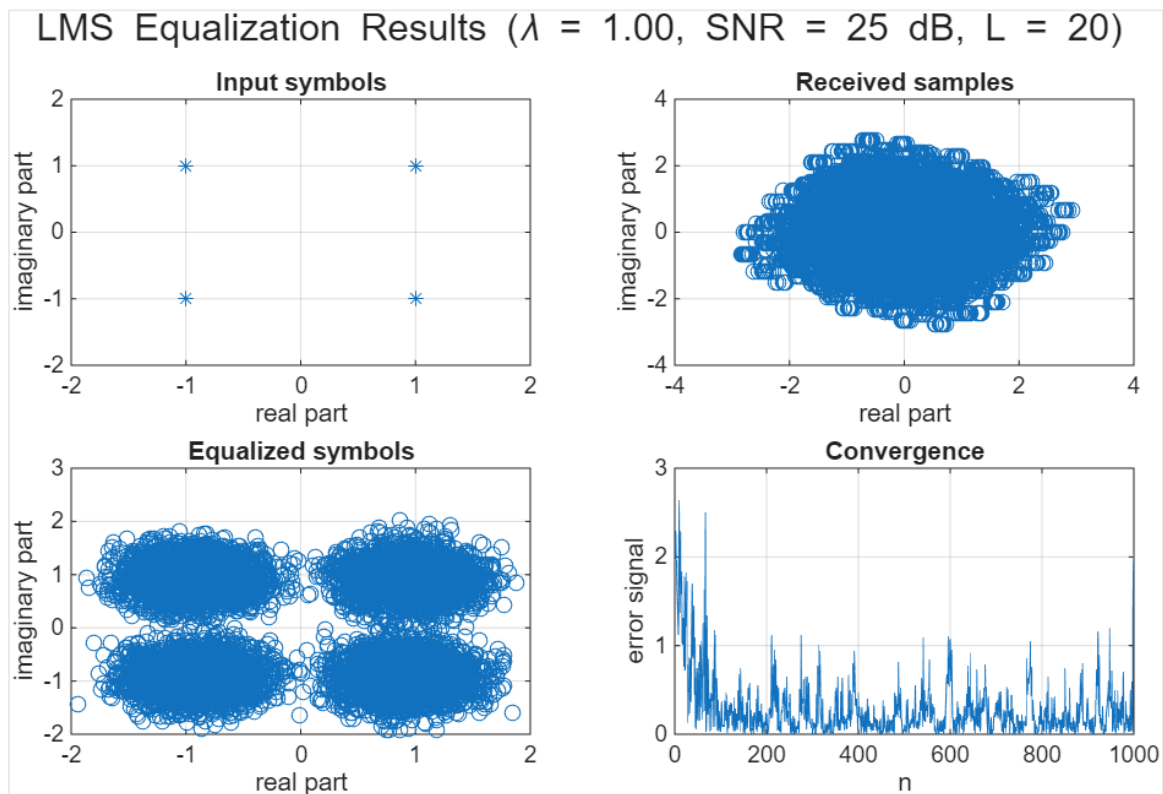


Figure 2. LMS Equalization Results ( $\lambda = 1.00$ , SNR = 25,  $L = 20$ )

Input symbols are shown in the Top left subplot, demonstrating the transmitted QPSK constellation. The symbols take only four ideal values  $\{\pm 1 \pm j\}$ , so the points appear at four fixed locations with no spread.

Received sample after transmission in the top right through the 5-tap channel and addition of noise, the received signal becomes a mixture of the current symbol and neighboring symbols due to inter-symbol interference (ISI). Therefore, the received constellation is no

longer four discrete points; instead, it forms a dense cloud centered around the origin. The widespread indicates that both channel distortion (ISI) and noise significantly affect the received samples before equalization.

Equalized symbols in the bottom-left show what happens after applying the LMS equalizer ( $L = 20$  taps). The constellation collapses back into four distinct clusters, showing that the equalizer has learned an approximate inverse response of the channel and has successfully reduced most of the ISI. The clusters are not single points but have noticeable thickness as noise is still present and cannot be fully removed and LMS has small steady-state fluctuations due to its fixed step size  $\mu$ . Overall, the separation between the four clusters confirms that equalization enables reliable QPSK detection.

Convergence plot shows the LMS training error (error signal magnitude/power) versus iteration index  $n$ . At the start, the error is high because the equalizer coefficients are initialized and the output does not match the training symbols. As adaptation proceeds, the error decreases quickly during the first part of the training and then reaches a low steady level. The remaining fluctuations and occasional spikes are expected because the input and noise are random; LMS continues updating with a constant  $\mu$ , so it does not settle to a perfectly constant error but oscillates around the optimum. The low error level after convergence is consistent with the low measured SER.

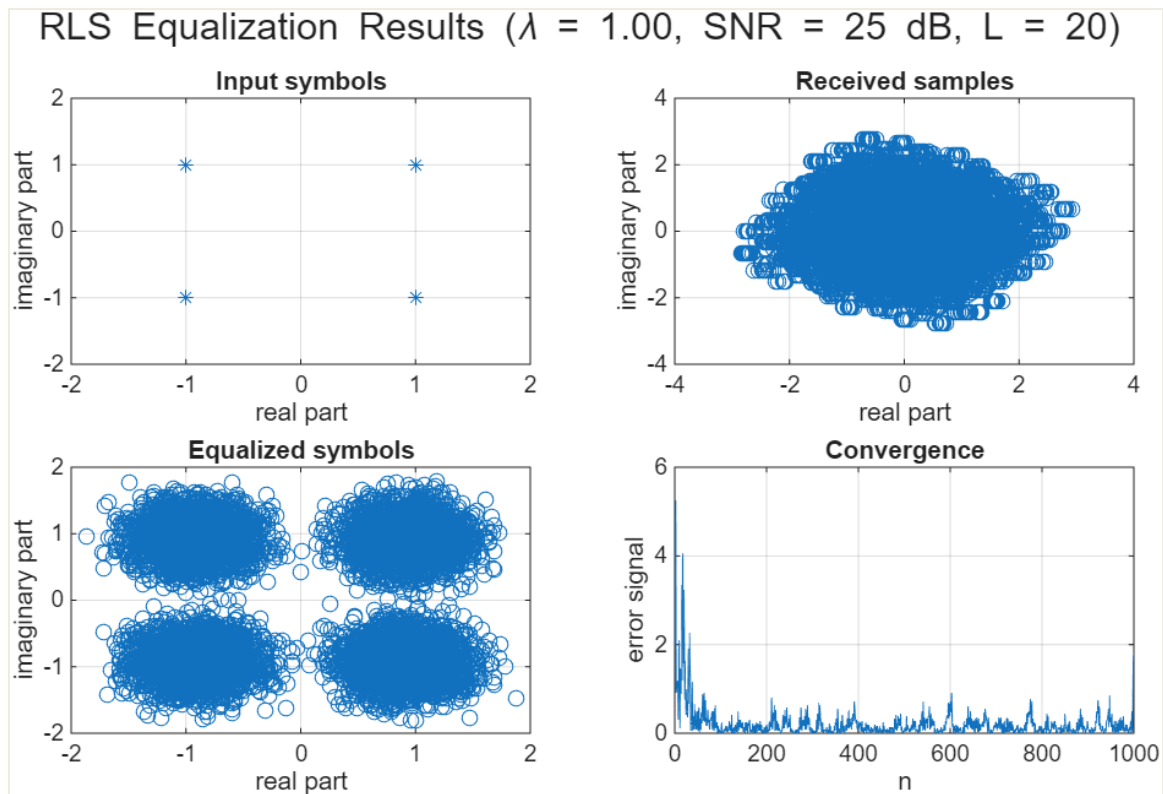


Figure 3. RLS Equalization Results ( $\lambda = 1.00$ , SNR = 25,  $L = 20$ )

Input symbols and received samples shown at the top left and top right are same as previous.

The bottom left plot shows the equalized symbols after applying the RLS equalizer ( $L = 20$  taps), the received cloud is mapped back into four separated clusters, indicating that RLS has effectively learned an inverse response of the channel and removed most of the ISI. Compared to the samples received, the equalized constellation is much more structured and clearly resembles QPSK. The remaining cluster thickness is mainly caused by the residual noise and small estimation errors, but the separation between clusters is sufficient for reliable symbol decisions.

The convergence plot at the bottom right shows the training error quickly dropping to a low level within the early part of the training interval. This fast error reduction is a characteristic advantage of RLS, since it uses a least-squares-type update that exploits correlation information from the received data. After convergence, the error remains near a low steady-state level with small fluctuations caused by noise. The fast settling and low error floor indicate stable equalizer adaptation for  $\lambda = 1$ .

#### **Comparison between LMS & RLS:**

**Convergence point of view:** From the learning curves (training error  $|e[n]|^2$  vs iteration), RLS converges significantly faster than LMS. RLS reduces the error sharply within the early part of the training sequence because it uses correlation information from past samples whereas LMS converges more gradually since it updates weights using a simple gradient step controlled by  $\mu$ . After convergence, LMS also shows slightly larger steady-state fluctuations due to its fixed step size, while RLS generally reaches a lower error level and stabilizes earlier.

**SER point of view:** After training, both algorithms achieve low SER, meaning both equalizers successfully mitigate most of the channel ISI at SNR = 25 dB and  $L = 20$  and  $\lambda=1$ .

LMS SER: 0.0077

RLS SER: 0.0012

**RLS gives the lower SER**, consistent with its faster convergence and more accurate final coefficient estimate for a static channel.

**Task 2:** Set  $\lambda = 0.98$  and repeat Task 1.

LMS SER: 0.0017

RLS SER: 0.0022

### LMS Equalization Results ( $\lambda = 0.98$ , SNR = 25 dB, L = 20)

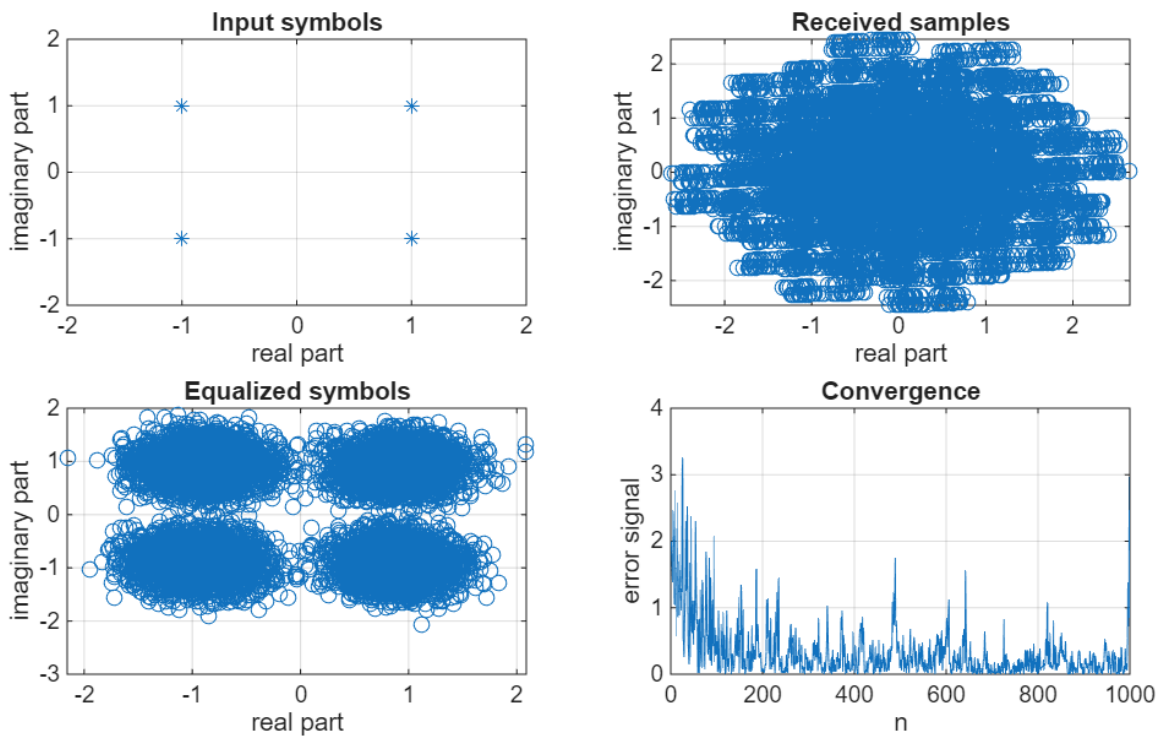


Figure 4. LMS Equalization Results ( $\lambda = 0.98$ , SNR = 25, L = 20)

### RLS Equalization Results ( $\lambda = 0.98$ , SNR = 25 dB, $L = 20$ )

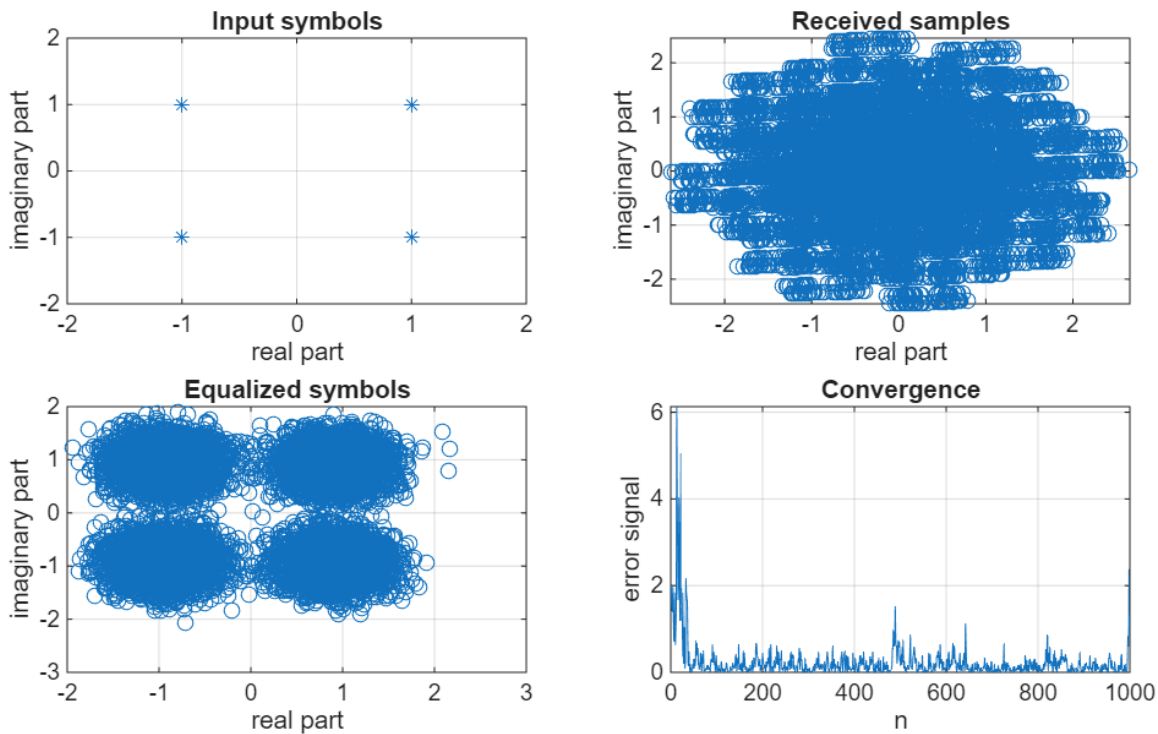


Figure 5. RLS Equalization Results ( $\lambda = 0.98$ , SNR = 25,  $L = 20$ )

When the forgetting factor is reduced from  $\lambda = 1.00$  to  $\lambda = 0.98$ , the main change is in the RLS behavior: with  $\lambda = 1.00$ , RLS uses all past training samples with equal weight, so it typically converges very quickly and reaches a very low steady error; with  $\lambda < 1$ , RLS applies exponential forgetting, giving more weight to recent samples and “discarding” older information. In a static channel this can slightly increase sensitivity to noise and lead to more weight jitter, which is visible as a somewhat less smooth steady-state error floor and a slightly worse equalized constellation compared to  $\lambda = 1.00$ . This effect also appears in the SER results: for  $\lambda = 0.98$ , both algorithms still equalize successfully (four QPSK clusters are clearly formed), but the final detection performance becomes closer and may even flip due to the increased adaptation noise in RLS; in the simulation LMS SER = 0.0017 while RLS SER = 0.0022, meaning LMS produced fewer symbol errors even though RLS still shows fast initial convergence. Overall, changing from  $\lambda = 1.00$  to 0.98 mainly makes RLS more tracking-oriented (useful for time-varying channels) but can slightly reduce its advantage in a static channel, while LMS is essentially unchanged because it does not use a forgetting factor.

#### Comparison between LMS & RLS:

**Convergence point of view:** From the learning curves (training error  $|e[n]|^2$  vs iteration), RLS still converges significantly faster than LMS. RLS reduces the error sharply within the early part of the training sequence because it uses correlation information from past samples,



whereas LMS converges more gradually since it updates weights using a simple gradient step controlled by  $\mu$ . However, when  $\lambda$  is reduced below 1, RLS applies a forgetting factor, giving more weight to recent samples. In a static channel this can make the RLS weight estimate slightly more sensitive to noise, so the steady-state error may show small additional fluctuations compared to the  $\lambda = 1$  case. LMS behavior is essentially unchanged because it does not use  $\lambda$ , and its steady-state misadjustment is mainly determined by  $\mu$ .

**SER point of view:** After training, both algorithms achieve low SER, meaning both equalizers successfully mitigate most of the channel ISI at SNR = 25 dB and L = 20 with  $\lambda = 0.98$ .

LMS SER: 0.0017

RLS SER: 0.0022

In this run, LMS gives a slightly lower SER than RLS. This can happen because  $\lambda < 1$  causes RLS to forget older data, which is useful for tracking time-varying channels but can slightly degrade the final estimate in a time-invariant channel. The difference is still small, and both methods produce clearly separated QPSK clusters after equalization.

## Conclusion

In this lab, adaptive channel equalization was studied for QPSK transmission over a 5-tap multipath channel at SNR = 25 dB using a 20-tap smoothing-type equalizer. Both LMS and RLS successfully mitigated the inter-symbol-interference, as confirmed by the received constellation collapsing into four well-separated QPSK clusters after equalization and by the low measured SER values. In terms of convergence, RLS consistently showed much faster reduction of the training error than LMS, reaching a low steady-state error within a shorter number of iterations due to its least-squares-based update. In terms of symbol detection performance, RLS provided better SER for  $\lambda = 1.00$  (LMS SER = 0.0077, RLS SER = 0.0012), matching its stronger convergence and lower steady-state error in a static channel. When the forgetting factor was reduced to  $\lambda = 0.98$ , the RLS algorithm became more sensitive to recent noisy samples, which reduced its advantage in this static scenario and resulted in a slightly higher SER than LMS (LMS SER = 0.0017, RLS SER = 0.0022). Overall, the results show that RLS offers faster convergence and strong performance for  $\lambda = 1$ , while using  $\lambda < 1$  improves tracking capability but can slightly degrade steady-state performance in a time-invariant channel; LMS remains simpler and can achieve comparable SER when properly tuned.

## References

- [1] S. M. Kay, Fundamentals of Statistical Signal Processing: Estimation Theory. Prentice Hall International, 1993.
- [2] K. P. Murphy, Machine Learning: A Probabilistic Perspective. Cambridge, MA, USA: MIT Press, 2012, Sec. 15.2.